
NVIDIA-CERTIFIED PROFESSIONAL

OpenUSD Development Practice Test

60 questions ■ 120 minutes ■ weighted by official exam domains

Difficulty	Mixed
Seed	4217 (reproducible)
Passing target	70% or higher

This practice test mirrors the question style, multi-select format, and domain weighting of the official NVIDIA-Certified Professional: OpenUSD Development exam. Questions and answer key are generated from a curated bank grounded in the Learn OpenUSD curriculum. Not affiliated with or endorsed by NVIDIA; for study use only.

Instructions

This test contains **60 questions** distributed across the eight exam domains by their official weighting. Each question is single-choice (choose one) unless it is explicitly marked (*Select two options.*) or (*Select three options.*), in which case you must select exactly that many answers for credit. Code questions present one or more USDA layers; reason through composition using **LIVERPS** strength order (Local, Inherits, Variants, rElocates, References, Payloads, Specializes). The **Answer Key & Explanations** section begins after the final question.

Domain distribution: Composition 14 · Data Exchange 9 · Pipeline Development 9 · Data Modeling 7 · Debugging and Troubleshooting 6 · Content Aggregation 6 · Visualization 5 · Customizing USD 4

Real exam: 60-70 questions, 120 min, Professional level. Weighting follows the official NVIDIA blueprint.

Question 1

Domain: Composition *(Select three options.)*

Which of the following edits can a variant within a variant set author on the prim and its descendants? (Select three.)

- A. Permanently remove other composition arcs authored outside the variant context.
- B. Author new composition arcs, such as a reference to a different material layer.
- C. Sparsely override property values on the prim or any descendant prim.
- D. Rename the prim itself to a different path within the same stage.
- E. Define entirely new descendant prims that exist only when that variant is active.

Source: Learn OpenUSD: Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/variant-sets/what-are-variant-sets.html>

Question 2

Domain: Data Exchange

A completed asset must be shipped to an external partner as one self-contained unit with its USD file and all referenced textures included. Which native OpenUSD file format is designed for exactly this atomic, read-only delivery?

- A. USDZ, because it is an uncompressed zip archive that bundles all necessary assets together.
- B. USD, because it can be either ASCII or binary without breaking references.
- C. USDA, because it is human-readable and easy to inspect after delivery.
- D. USDC, because it uses compression and memory mapping for efficient loading.

Source: Learn OpenUSD: OpenUSD File Formats — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/usd-file-formats.html>

Question 3

Domain: Composition

How do payloads differ from references with respect to loading behavior and composition strength in LIVERPS?

- A. Payloads can be loaded and unloaded on demand and are weaker than references in LIVERPS.
- B. Payloads load eagerly on stage open and are weaker than references in LIVERPS.
- C. Payloads can be loaded and unloaded on demand and are stronger than references in LIVERPS.
- D. Payloads load eagerly on stage open and are stronger than references in LIVERPS.

Source: Learn OpenUSD: Payloads — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/what-are-payloads.html>

Question 4

Domain: Pipeline Development *(Select three options.)*

A studio is designing layer stacks to model parallel workstreams for an asset and for a distributed simulation pipeline. Which three statements reflect the correct guidance? (Select three.)

- A. Computational workstream layers must always be static; dynamic partitioning across machines is not supported in a layer stack.
- B. Layer stacks can grow procedurally over time to accumulate simulation steps, replacing the need for a versioning system.
- C. Each workstream layer should author only its unique contributions sparsely, avoiding needless overwrites of other workstreams.
- D. A geometry layer and a shading layer can be sublayered together so artists iterate independently without blocking each other.
- E. Keeping layer stacks simple and manageable is recommended because there is a cost associated with resolving and opening each layer.

Source: Learn OpenUSD: Modeling Workstreams With Layer Stacks — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/workstreams/modeling-workstreams.html>

Question 5

Domain: Composition

An asset uses an inherit arc targeting unencapsulated class `/_cube_asset`. A downstream stage overrides `/_cube_asset` in a stronger layer. Which statement correctly describes the scope of that override?

- A. The override converts `/_cube_asset` from a class prim to a def prim in `Scenario_A`.
- B. The override is ignored because inherit arcs cannot be overridden by opinions in stronger layers.
- C. The override affects `Scenario_A`'s composition only; other stages referencing the asset are not affected.
- D. The override propagates globally to all stages that reference the same asset.

Source: Learn OpenUSD: Inherits — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/inherits-specializes/what-is-inherits.html>

Question 6

Domain: Content Aggregation

Primvar values are authored directly on several instanceable prims that all share one prototype, giving each prim a different color. What is the effect on the prototype count, and why?

- A. One new prototype is created for the group of prims that received overrides, leaving the unmodified prims on the original prototype.
- B. No new prototypes are created because OpenUSD merges primvar overrides into the existing prototype silently.
- C. One new prototype is created per distinct primvar value because each colored variant requires a unique subgraph.
- D. No new prototypes are created because primvar opinions live on the instanceable prims, not inside the shared subgraph.

Source: Learn OpenUSD: Hierarchical Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-hierarchical-refinement.html>

Question 7

Domain: Pipeline Development

An asset organizes all mesh prims under a 'Geometry' scope and all material prims under a 'Looks' scope, both children of the asset's 'defaultPrim'. What does this convention primarily establish?

- A. It satisfies the encapsulation requirement by ensuring material prims are nested under the same prim as the geometry they bind.
- B. It defines jurisdictions that separate modeler contributions from surfacer contributions, making it clear which prims each workstream owns.
- C. It establishes kind metadata boundaries so that traversal prunes at the Geometry and Looks scopes instead of at the defaultPrim.
- D. It marks the geometry and material prims as components within the model hierarchy so pipeline tools can discover them efficiently.

Source: Learn OpenUSD: Exercise: Organizing Prim Hierarchy — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-structure-principles/organizing-prim-hierarchy.html>

Question 8

Domain: Debugging and Troubleshooting

Two sublayers both author customData on the same prim. The stronger layer sets keys 'source' and 'status'. The weaker layer sets keys 'source' and 'owner'. What does the composed customData contain?

- A. 'source' from the stronger layer, 'status' from the stronger layer, and 'owner' from the weaker layer.
- B. The keys from the stronger layer — 'source' and 'status' — because dictionaries follow a winner-takes-all rule across the full dictionary.
- C. A composition error, because two layers may not author overlapping customData keys on the same prim.
- D. The keys from the weaker layer — 'source' and 'owner' — because dictionaries accumulate entries from the weakest layer upward.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 9

Domain: Customizing USD

A schema maintainer is modifying an existing typed schema. Which change requires creating a new schema version according to the USD schema versioning guidelines?

- A. Updating a token attribute's allowedTokens field to add a new previously disallowed token
- B. Changing the displayName or doc metadata on an existing property
- C. Adding a new attribute with a fallback value that has no behavioral effect on prims lacking that attribute
- D. Changing the fallback value of an existing attribute in a way that affects prim behavior for downstream consumers

Source: OpenUSD: Schema Versioning — https://openusd.org/release/wp_schema_versioning.html

Question 10

Domain: Composition

An asset file 'asset.usd' has a root prim 'World'. Inside 'World' there is a 'Cube' prim with a material binding that targets a 'Material' prim defined under a sibling 'Looks' scope at the same level as 'World', not nested beneath it. After the asset is referenced into a scene, the material binding stops working. What is the root cause?

- A. The Looks scope is unencapsulated — it lives outside the referenced root prim hierarchy and is not grafted into the new scene
- B. USD requires material prims to be direct children of the geometry prim they bind; sibling scopes are invalid for binding
- C. Material bindings are not preserved across reference arcs; the binding relationship must be re-authored on the destination prim
- D. The defaultPrim metadata points to World, so prims outside World are treated as the entry point and excluded from references

Source: Learn OpenUSD: What Is Encapsulation? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/what-is-encapsulation.html>

Question 11

Domain: Debugging and Troubleshooting

A pipeline developer wants to understand whether USD caches composition results or resolved attribute values at stage load time. Which statement correctly describes the caching model?

- A. Neither composition nor value resolution is cached; all data is read directly from layers on each access.
- B. Value resolution is cached per attribute, but composition is recomputed on every property query.
- C. Both composition and value resolution results are cached at load time for maximum query speed.
- D. Composition is cached at the prim level, but value resolution is not pre-calculated.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 12

Domain: Composition

A prim is authored with the class specifier. What effect does the class specifier have on rendering and default stage traversal?

- A. Hydra renders the prim at reduced resolution and default traversals exclude it from physics simulation.
- B. Hydra does not render the prim and default stage traversals do not visit it.
- C. The prim is marked read-only and other composition arcs are prevented from targeting it.
- D. Default traversals visit the prim twice: once entering and once exiting its hierarchy.

Source: Learn OpenUSD: Specifiers — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/specifiers.html>

Question 13

Domain: Content Aggregation

A PointInstancer has hundreds of scattered boxes. Some boxes fell out of position and must be animated on and off over time; other boxes must be permanently pruned and the pruning must support sparse list-editing. Which property

drives each requirement?

- A. `inactivelds` animates instances on and off over time; `invisiblelds` prunes them permanently with list-editing support.
- B. `invisiblelds` hides instances over time; `protoIndices` removes them permanently with list-editing support.
- C. `visibility` animates instances on and off over time; `invisiblelds` prunes them permanently with list-editing support.
- D. `invisiblelds` animates instances on and off over time; `inactivelds` prunes them permanently with list-editing support.

Source: Learn OpenUSD: Refining Point Instances — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-point-instances.html>

Question 14

Domain: Data Exchange (Select three options.)

Which three statements correctly identify benefits of the data extraction phase in a two-phase data exchange pipeline? (Select three.)

- A. Keeping the extracted stage faithful to the source simplifies round-tripping of data back to the original format.
- B. The extraction phase applies user export options so that end users receive a tailored output without needing a separate transformation step.
- C. The extraction phase automatically generates a compliant `defaultPrim` and resolves `upAxis` conflicts between composed layers.
- D. The extracted stage provides a common foundation on which all subsequent data transformation steps can be built.
- E. The extracted output mirrors the source format structure, improving legibility and making it easier to debug for developers and end users.

Source: Learn OpenUSD: What Is Data Extraction? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/what-is-data-extraction.html>

Question 15

Domain: Data Modeling (Select two options.)

When querying a resolved attribute value in OpenUSD, which statements about attribute value resolution and time codes are correct?

- A. Calling `Get()` with no time code returns the first authored time sample rather than the authored default value.
- B. Calling `Get()` with no time code is equivalent to `Get(Usd.TimeCode.Default())` and returns the authored default value.
- C. Value resolution results are cached at the prim level in the same way composition results are cached.
- D. If no value is authored for an attribute, value resolution returns the nearest authored value from an ancestor prim.
- E. Using `Usd.TimeCode.EarliestTime()` returns the first authored time sample value when animation exists.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 16

Domain: Pipeline Development (Select two options.)

A studio is organizing assets into parallel workstreams using OpenUSD layer stacks. Which practices are recommended?

- A. Computational workstreams should remain static and consistent from one evaluation of the stage to the next
- B. Simple assets should be split into separate workstream layers up front so parallelism is available later
- C. Layer stacks should serve as the asset versioning mechanism, with each version captured as a sublayer
- D. Each workstream should author only its unique contributions sparsely rather than overwriting other workstreams' work
- E. Layer stacks should be kept simple, avoiding workstreams that grow procedurally, since each layer has a resolve-and-open cost

Source: Learn OpenUSD: Modeling Workstreams With Layer Stacks — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/workstreams/modeling-workstreams.html>

Question 17

Domain: Composition

A prim specializes a class defining an attribute fallback of 30. The prim also individually authors its own value of 20. The class fallback is later changed to 40. What value does the prim resolve to?

- A. 20, because any individually authored opinion on the prim is stronger than the specializes fallback.
- B. A composition conflict arises and the prim falls back to the schema default of 0.
- C. 40, because the specializes arc always propagates the latest class value to all destination prims.
- D. 30, because specializes retains the original fallback value at the time the arc was authored.

Source: Learn OpenUSD: Specializes — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/inherits-specializes/what-is-specializes.html>

Question 18

Domain: Content Aggregation

The mantra for scenegraph instancing is "explicit instances, implicit prototypes." What does this mean in OpenUSD?

warehouse.usda

```
#usda 1.0
(
    defaultPrim = "Warehouse"
)
def Xform "Warehouse"
{
    def "RobotArm_01" (
        references = @./RobotArm.usd@
        instanceable = true
    )
    {
    }
```



```
def "RobotArm_02" (
    references = @./RobotArm.usd@
    instanceable = true
)
{
}
}
```

- A. OpenUSD explicitly creates one prototype per prim type and implicitly assigns every prim of that type to it
- B. You explicitly list each instance path, and OpenUSD implicitly disables overrides on the whole stage
- C. You explicitly tag candidate prims with `instanceable = true`, and OpenUSD derives the prototypes implicitly from the authored composition arcs
- D. You explicitly author each prototype as a relationship target, and OpenUSD implicitly chooses which instances to share it

Source: Learn OpenUSD: Introduction to Scenegraph Instancing — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/authoring-scenegraph-instancing/scenegraph-instancing-intro.html>

Question 19

Domain: Composition

A USD file has a Cube under `/World` and a material scope `/Looks` at the same level as `/World`, not nested beneath it. The Cube binds to `/Looks/RedMat`. After referencing `/World` as source prim, what happens to the binding?

- A. The binding resolves correctly because USD automatically remaps sibling-scope paths during referencing.
- B. The binding stops working because `/Looks` is not encapsulated within the referenced prim hierarchy.
- C. The binding is promoted to the root layer of the referencing stage so it stays reachable.
- D. The binding is silently discarded and no warning is emitted by the composition engine.

Source: Learn OpenUSD: What Is Encapsulation? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/what-is-encapsulation.html>

Question 20

Domain: Pipeline Development

A technical director tags a leaf Mesh prim deep in an asset's hierarchy with `'kind = "component"'`. According to model hierarchy rules, what structural problem does this create?

- A. A leaf Mesh tagged as a component violates the nesting rule for component models, because a Mesh is itself treated as a kind of component.
- B. A leaf Mesh tagged as a component is invalid because the Mesh typeName conflicts with the component kind and USD will silently strip the metadata.
- C. A leaf Mesh tagged as a component signals that the model hierarchy is too deep, because a gprim at component level is the explicit sign of excessive depth.
- D. A leaf Mesh tagged as a component requires its ancestor prims to carry `kind = component` to maintain a valid hierarchy.

Source: Learn OpenUSD: Model Hierarchy Considerations — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/model-hierarchy-considerations.html>

Question 21

Domain: Data Exchange

When authoring USD materials during import, which API pairing correctly creates a preview shader and binds it to geometry?

- A. Author a raw shininess attribute on the Mesh and bind it with `UsdSkelBindingAPI`
- B. Create a Material prim and rely on namespace prefixes to auto-bind to the nearest Mesh
- C. Build a `UsdPreviewSurface` via `UsdShade` and bind it with `UsdShadeMaterialBindingAPI`
- D. Build a `UsdPreviewSurface` via `UsdGeom` and bind it with `UsdGeomBindingAPI`

Source: Learn OpenUSD: Exercise: Extracting Materials — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-materials.html>

Question 22

Domain: Pipeline Development

A pipeline tool must compose an entire stage down to a single layer for delivery, removing composition arcs and proprietary file-format dependencies. Which operation is correct, and what distinction matters?

- A. `UsdUtilsFlattenLayerStack` composes the whole stage into one layer; `UsdStage.Flatten` merges the sublayers of the root layer stack
- B. `SdfChangeBlock` collapses the stage's composition arcs into one layer when the block closes; this differs from flattening a layer stack
- C. Setting the edit target to the root layer bakes composed opinions into that layer; this differs from flattening the full stage
- D. `UsdStage.Flatten` composes the whole stage to one layer and removes arcs; this differs from flattening a single layer stack

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

Question 23

Domain: Composition

A developer wants three identical sphere prims at sibling prim paths in one scene. They add the sphere layer as a sublayer three times. Why does this fail to produce three distinct sibling prims?

- A. Sublayering a layer three times causes USD to treat all three as a single merged opinion.
- B. Sublayers include content without any prim-path remapping, so all three copies resolve to the same prim path.
- C. Sublayers do not support prim definitions; only the over specifier is permitted inside a sublayer.
- D. Sublayers can only be added once per root layer before USD raises an error.

Source: Learn OpenUSD: References — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/what-are-references.html>

Question 24

Domain: Visualization

A UsdLuxDistantLight is authored with only xformOp:rotateXYZ = (45, 0, 0) and no other transforms. Before any transform is applied, along which local axis does the DistantLight emit by default?

example.usda

```
def DistantLight "SunLight"
{
    float3 xformOp:rotateXYZ = (45, 0, 0)
    uniform token[] xformOpOrder = ["xformOp:rotateXYZ"]
}
```

- A. Along the local -Z axis, representing emission toward the default gaze direction
- B. Along the local +X axis, consistent with the horizontal orientation of a sun light
- C. Along the local +Z axis, representing emission away from the default viewing plane
- D. Along the local +Y axis, consistent with the Y-up default coordinate system

Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>

Question 25

Domain: Content Aggregation (Select three options.)

A scene uses broadcasted refinement via a specialize arc so that all boxes on one pallet share a common class namespace target. A single reference is added to that class target. Which three statements correctly describe the outcome? (Select three.)

- A. The prim count increases because the new arc and its referenced content introduce additional prims into the stage.
- B. No new prototypes are created because broadcasted refinement, like hierarchical primvar refinement, never generates additional prototypes.
- C. The total prim count decreases because deinstancing is avoided and fewer per-instance overrides are needed.
- D. The instance count stays the same because instancing remains active on all boxes throughout the operation.
- E. A new prototype is created for the affected boxes because their composed subgraph now differs from unmodified boxes.

Source: Learn OpenUSD: Broadcasted Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-broadcasted-refinement.html>

Question 26

Domain: Data Exchange

In the two-phase OpenUSD data exchange approach, what is the stated goal of the extract phase?

- A. To prune data irrelevant to a workflow and optimize the output for a specific end client.
- B. To apply user export options and restructure content to fit organization-specific asset conventions.
- C. To translate data to OpenUSD as directly as possible, mapping source concepts to USD concepts to preserve fidelity.
- D. To validate the output stage against compliance rules and flag missing metadata before delivery.

Source: Learn OpenUSD: What Is Data Exchange? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-exchange/what-is-data-exchange.html>

Question 27

Domain: Pipeline Development

In the reference/payload pattern, a primvar and a variant set are moved from the contents layer to the interface layer above the payload. What is the primary benefit of this lofting?

- A. Lofting removes the fields from the payload so they contribute no opinions to the composed stage.
- B. Lofting promotes the fields to the root layer of the stage so they override downstream sublayer opinions.
- C. Lofting makes the fields discoverable and usable by consumers even when the payload is unloaded, improving performance in unloaded contexts.
- D. Lofting converts the fields from payload-local opinions into global layer metadata that all stages can read.

Source: Learn OpenUSD: Exercise: Lofting Primvars — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/reference-payload-pattern/lofting-primvars.html>

Question 28

Domain: Composition

A prim references an asset using an inner Cube as the source prim, rather than the asset's root World prim. The Cube has a material binding to a material defined under World. What is the result?

- A. Both the Cube and its material compose, because relationship targets are re-pathed to follow their prim across the reference
- B. The Cube's geometry composes but the material binding is dropped with a warning, because the bound material lies outside the scope of the reference
- C. Nothing composes onto the prim, because a reference whose source prim is not the layer's defaultPrim fails to resolve and is skipped
- D. The material composes but the geometry is dropped, because relationship targets take precedence over prim specs during grafting

Source: Learn OpenUSD: Exercise: Experimenting With Encapsulation — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/experimenting-with-encapsulation.html>

Question 29

Domain: Data Exchange

A studio is building USD exporters for three different DCC applications. A developer asks whether the extraction and transformation code can be shared across all three exporters. What does the two-phase guidance say about reusability?

- A. Transformation code is largely unique to each target workflow, while extraction code can be designed for reuse across multiple exporters.
- B. Extraction code is largely unique to each source format, while transformation code can be designed for reuse across multiple exporters.

- C. Both extraction and transformation code are largely unique to each source format and must be reimplemented per exporter.
- D. Both extraction and transformation code can be designed for reuse because they both operate on OpenUSD data after the initial parse.

Source: Learn OpenUSD: What Is Data Transformation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/what-is-data-transformation.html>

Question 30

Domain: Debugging and Troubleshooting

A developer calls `SetActive(False)` on `/World/Box`, which has children `/World/Box/Geometry` and `/World/Box/Materials`. What does the next call to `stage.Traverse()` visit?

- A. `/World/Box` raises a Python exception because deactivated prims cannot be accessed via `GetPrimAtPath`.
- B. `/World/Box` is skipped but its active descendants `/World/Box/Geometry` and `/World/Box/Materials` are still visited.
- C. `/World/Box` and its descendants remain in the traversal but their attributes resolve to empty values.
- D. `/World/Box` is skipped and all of its descendants are also excluded from the traversal.

Source: Learn OpenUSD: Active and Inactive Prims — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/active-inactive-prims.html>

Question 31

Domain: Data Modeling

A two-quad mesh has 6 points and 2 faces. A `primvars:displayColor` is authored with 'constant' interpolation. How many color values must its array contain?

- A. 1 — one value for the entire prim
- B. 6 — one value per vertex
- C. 2 — one value per face
- D. 8 — one value per face-vertex entry

Source: Learn OpenUSD: Primvars — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/primvars.html>

Question 32

Domain: Debugging and Troubleshooting

An attribute has an authored default of `(1.5,1.5,1.5)` and time samples starting at time code 60 with value `(2.5,2.5,2.5)`. The stage `startTimeCode` is 1. What does `attr.Get()` return, and `attr.Get(Usd.TimeCode(1))`?

- A. `attr.Get()` returns `(2.5, 2.5, 2.5)` and `attr.Get(Usd.TimeCode(1))` returns `(2.5, 2.5, 2.5)`.
- B. `attr.Get()` returns `(1.5, 1.5, 1.5)` and `attr.Get(Usd.TimeCode(1))` returns `(2.5, 2.5, 2.5)`.
- C. `attr.Get()` returns `(2.5, 2.5, 2.5)` and `attr.Get(Usd.TimeCode(1))` returns `(1.5, 1.5, 1.5)`.
- D. `attr.Get()` returns `(1.5, 1.5, 1.5)` and `attr.Get(Usd.TimeCode(1))` returns `(1.5, 1.5, 1.5)`.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 33

Domain: Data Modeling

A USD file does not explicitly author `metersPerUnit`. According to OpenUSD's fallback behavior, what linear-unit scale does the runtime assume?

- A. 0.001 — one unit equals one millimeter by default
- B. 1.0 — one unit equals one meter by default
- C. 0.01 — one unit equals one centimeter by default
- D. 0.3048 — one unit equals one foot by default

Source: Learn OpenUSD: Units in OpenUSD — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/units.html>

Question 34

Domain: Data Exchange

Which statement correctly describes the reusability difference between extraction code and transformation code across multiple exporters?

- A. Extraction code and transformation code are both reusable across source formats, since every exporter emits OpenUSD data
- B. Extraction code and transformation code are both unique per source format, since each exporter targets a different data model
- C. Transformation code is unique to each source format, while extraction code is reusable since all sources parse into OpenUSD
- D. Extraction code is largely unique per source format, while transformation code can be shared across exporters since it operates on OpenUSD data

Source: Learn OpenUSD: What Is Data Transformation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/what-is-data-transformation.html>

Question 35

Domain: Visualization

An artist wants a geometry mesh prim to emit light during rendering without defining a separate light prim elsewhere in the hierarchy. Which mechanism in UsdLux makes this possible?

- A. Defining a `UsdLuxGeometryLight` typed schema prim as a child of the mesh to inherit its shape
- B. Connecting the mesh prim's material surface output to a `UsdLuxSphereLight` inputs:color port
- C. Applying a UsdLux API schema such as `MeshLightAPI` directly to the existing mesh prim
- D. Setting the mesh prim's purpose attribute to 'render' so Hydra treats it as a light emitter

Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>

Question 36

Domain: Content Aggregation

Three prims each have 'instanceable = true' set, but none of them carry any composition arcs. How many implicit prototypes does OpenUSD generate, and why?

- A. Zero prototypes are generated because instanceable metadata is restricted to assembly prims and has no effect elsewhere.
- B. Zero prototypes are generated because instancing requires composition arcs to derive prototypes from.
- C. Three prototypes are generated, one per instanceable prim, because the instanceable flag itself drives prototype creation.
- D. One prototype is generated because all three prims share the same local layer stack.

Source: Learn OpenUSD: Introduction to Scenegraph Instancing — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/authoring-scenegraph-instancing/scenegraph-instancing-intro.html>

Question 37

Domain: Composition

A root layer lists sublayerB.usda first and sublayerA.usda second. Both sublayers author a value for the same attribute on the same prim. Which sublayer's opinion is resolved as the composed value?

- A. An average of both sublayers' values, because USD blends conflicting local opinions.
- B. The root layer's own opinion, because the root layer always overrides any sublayer.
- C. sublayerA.usda, because the last entry in the subLayers list is treated as strongest.
- D. sublayerB.usda, because the first entry in the subLayers list is treated as strongest.

Source: Learn OpenUSD: Sublayers — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/sublayers/what-are-sublayers.html>

Question 38

Domain: Content Aggregation

Two instanceable prims both reference the same asset file. The first has no additional composition arcs; the second also has a variant selection authored on it. How many prototypes does OpenUSD generate for these two prims?

example.usda

```
#usda 1.0
def Xform "Warehouse"
{
    def "Robot_01" (
        references = @./Robot.usd@
        instanceable = true
    ) {}
    def "Robot_02" (
        references = @./Robot.usd@
        instanceable = true
        variants = {string color = "blue"}
    ) {}
}
```

- A. Zero prototypes are generated because variant selections prevent prototype sharing.
- B. Two prototypes are generated because the variant selection gives the second prim a distinct composed subgraph.
- C. Two prototypes are generated because each instanceable prim always receives its own prototype.
- D. One prototype is generated because both prims reference the same asset file.

Source: Learn OpenUSD: Refinement Using Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-variant-set-refinement.html>

Question 39

Domain: Data Exchange

An asset authored at `metersPerUnit = 1.0` (meters) is referenced into a scene with `metersPerUnit = 0.01` (centimeters) and no corrective transform is applied. According to OpenUSD's unit handling behavior, what is the result?

- A. The runtime automatically scales the referenced geometry so it appears at the correct real-world size in the scene.
- B. The geometry is composed at its raw numeric values without unit conversion, and it appears 100 times too small.
- C. The stage refuses to compose the reference because the `metersPerUnit` values do not match.
- D. The geometry is composed at its raw numeric values without unit conversion, and it appears 100 times too large.

Source: Learn OpenUSD: Units in OpenUSD — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/units.html>

Question 40

Domain: Data Exchange

A pipeline team wants to go beyond basic command-line USD compliance checks and needs a validation workflow with a Python API, automatic fixes for failed validations, and the ability to add custom rules. Which tool provides these capabilities?

- A. `usdcats`, because it can be used with file format plugins as a standalone converter with inspection support.
- B. `UsdGeomBBoxCache`, because it validates geometry bounding boxes and flags spatial inconsistencies across composed layers.
- C. Omniverse Asset Validator, because it extends `usdchecker` with a user interface, a Python API, automatic fixes, and custom rule support.
- D. `usdchecker`, because it validates a USD stage or USDZ package using a defined rule set from the command line.

Source: Learn OpenUSD: What Is Asset Validation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/asset-validation/what-is-asset-validation.html>

Question 41

Domain: Pipeline Development

An asset's material prim is defined at a path outside the 'defaultPrim' hierarchy. When the asset is referenced into a scene, what happens to that material binding?

- A. The material binding is silently remapped to the nearest in-scope material prim by the composition engine.
- B. The material binding continues to function because references pull the entire source layer, including prims outside the defaultPrim.
- C. The material binding fails when the reference also specifies a payload, but succeeds in reference-only contexts.
- D. The material binding target refers to a path outside the scope of the reference and is ignored, causing the binding to stop working.

Source: Learn OpenUSD: Exercise: Encapsulating Your Asset — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-structure-principles/encapsulating-your-asset.html>

Question 42

Domain: Debugging and Troubleshooting

A tool must traverse only prims that are both active and loaded. A developer writes 'predicate = Usd.PrimIsActive and Usd.PrimIsLoaded'. Why does this produce incorrect traversal behavior?

- A. The and keyword internally calls `__bool__` on each predicate object, which always returns True, causing the predicate to match every prim.
- B. The and keyword short-circuits and discards Usd.PrimIsLoaded, so only active prims are filtered.
- C. Traversal predicates must be combined with bitwise operators such as `&`, not with the boolean and keyword.
- D. The and keyword produces a set union rather than an intersection, so unloaded prims that are active are also visited.

Source: Learn OpenUSD: Stage Traversal — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/stage-traversal.html>

Question 43

Domain: Composition

A prim has no local opinion for an attribute. Competing opinions arrive via an inherit arc, a reference arc, and a specializes arc. Under LIVERPS, which arc's opinion is composed as the final value?

- A. The inherits arc, because inherits is ordered above variant sets, references, and specializes.
- B. The specializes arc, because specializes provides the fallback of last resort.
- C. The reference and inherits arc opinions are merged by averaging their values.
- D. The reference arc, because references are the most common composition arc.

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

Question 44

Domain: Data Modeling

A stage sets `timeCodesPerSecond = 24`. An attribute has a time sample at time code 48.0. How many seconds of playback after time code 0 does this sample represent?

- A. 1 second — time code 48 at 48 fps maps to 1 second
- B. 24 seconds — one time code per second at this rate

- C. 0.5 seconds — time code 48 scaled by the 24-fps factor yields 0.5
- D. 2 seconds — time code 48 divided by 24 equals 2 seconds

Source: Learn OpenUSD: Time Codes and Time Samples — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/timcodes-timesamples.html>

Question 45

Domain: Data Modeling

A developer authors an attribute with `custom=True` using `UsdPrim::CreateAttribute()`. What effect does the `custom=True` flag have on how OpenUSD internally processes that attribute?

- A. It registers the attribute in the schema registry so tools can discover it automatically
- B. It marks the attribute as informational metadata for developers but has no impact on how OpenUSD treats the attribute
- C. It prevents the attribute from being overridden by stronger composition opinions
- D. It restricts the attribute from carrying time samples and limits it to a default value only

Source: Learn OpenUSD: Custom Properties — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/custom-properties.html>

Question 46

Domain: Composition

A prim receives an opinion for radius from an inherited class, from a selected variant, and from a reference, with no local opinion present. Which opinion wins?

- A. The variant opinion, because variant selections override inherited class opinions
- B. The referenced opinion, because references resolve before inherits
- C. The inherited opinion, because inherits is stronger than variants and references
- D. The opinion in the strongest sublayer, regardless of which arc delivered it

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

Question 47

Domain: Visualization

OpenUSD ships with several render delegates. Which delegate provides the real-time OpenGL/Metal/Vulkan rendering used by `usdview` and many other tools out of the box?

- A. HdStorm — the interactive rasterization delegate included with the core distribution
- B. HdEmbree — the production ray-tracing delegate used by `usdview` by default
- C. HdTiny — the lightweight delegate optimized for interactive viewport rendering in production
- D. HdPrman — the RenderMan-based delegate that ships with the core OpenUSD distribution

Source: Learn OpenUSD: Hydra — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/hydra.html>

Question 48

Domain: Visualization

Hydra is described as having three main parts. Which set correctly names all three and their roles?

- A. Render index (provides scene information), scene delegate (tracks changes), render delegate (issues draw calls to the GPU)
- B. Scene delegate (creates geometry), shader graph (manages material connections), render backend (generates pixels)
- C. Scene proxy (provides scene information), render context (tracks changes), render plugin (creates the final image)
- D. Scene delegate (provides scene information), render index (tracks changes and manages the scene), render delegate (creates the final image)

Source: Learn OpenUSD: Hydra — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/hydra.html>

Question 49

Domain: Data Exchange

A proprietary CAD application keeps geometry in its own native format as the single source of truth and must compose directly into a USD stage without a pre-export step. Which OpenUSD integration mechanism uniquely enables this, and which capability makes it the right choice?

- A. A standalone converter, because it can be run as a batch process that translates the native format to USD at composition time.
- B. A DCC exporter, because it translates the application's runtime format to OpenUSD and supports bidirectional conversion.
- C. A DCC importer, because it translates data from the application's runtime format into OpenUSD structures on demand.
- D. A file format plugin, because the native file can be referenced, sublayered, or loaded as a payload into a USD stage without a prior conversion step.

Source: Learn OpenUSD: What Is Data Exchange? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-exchange/what-is-data-exchange.html>

Question 50

Domain: Data Modeling (Select two options.)

Which two of the following unit metadata fields are automatically reconciled by USD during composition, requiring no manual corrective transform from the pipeline? (Select two.)

- A. upAxis — USD automatically rotates geometry when upAxis values differ between layers
- B. kilogramsPerUnit — USD automatically adjusts mass values for physics simulations
- C. metersPerUnit — USD automatically applies a scale factor to referenced geometry
- D. timeCodesPerSecond applied via a layer offset — USD uses the layer offset to rescale time samples automatically
- E. timeCodesPerSecond — USD automatically scales time samples so animation timing remains correct

Source: Learn OpenUSD: Units in OpenUSD — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/units.html>

Question 51

Domain: Data Modeling

A pipeline must store temperature, humidity, and pressure sensor readings on a single prim without authoring a new schema. USD has no native struct type. Which approach is recommended?

- A. Use namespace-prefixed attributes: `acme:sensor:temperature`, `acme:sensor:humidity`, and `acme:sensor:pressure`
- B. Author each value into a `customData` dictionary entry under a shared sensor key
- C. Store the readings as a single string attribute using a structured text encoding
- D. Define a Scope child prim to hold each reading as its own child prim

Source: Learn OpenUSD: Custom Properties — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/custom-properties.html>

Question 52

Domain: Pipeline Development

A prim is marked `'kind = "component"'`. According to model hierarchy rules, which statement about this prim's descendants is correct?

- A. Component descendants must all carry `kind = subcomponent` so that downstream traversal can find material prims.
- B. Component descendants may include prims marked `kind = assembly` provided they are at least two levels below the component root.
- C. Component descendants may include prims marked `kind = group` to provide optional organizational grouping below the component boundary.
- D. Component descendants cannot include other prims marked `kind = component` because the component is the leaf of the model hierarchy.

Source: Learn OpenUSD: What Are Model Kinds? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/what-are-model-kinds.html>

Question 53

Domain: Pipeline Development

An asset exposes both a color variant set and a primvar on its entry point for downstream parameterization. How do the two approaches differ with respect to scenegraph instancing?

- A. Primvar overrides generate new prototypes for each distinct value authored, while variant selections share one prototype across all selections.
- B. Variant selections generate new prototypes for each distinct selection, while primvar overrides do not generate new prototypes.
- C. Variant selections and primvar overrides both share a single prototype regardless of how many different values are authored.
- D. Variant selections and primvar overrides both generate one new prototype per distinct parameterization value.

Source: Learn OpenUSD: What Is Asset Parameterization? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-parameterization/what-is-asset-parameterization.html>

Question 54

Domain: Debugging and Troubleshooting (Select two options.)

A teammate claims composition results are cached but value resolution is not. Which statements from the USD value-resolution model are correct?

- A. How data combines during value resolution is identical for every metadatum, regardless of its data type
- B. Value resolution is internally multi-threaded by USD, so opening many stages in parallel threads is the recommended way to scale
- C. GetPropertyStack should be used as the primary value-resolution path for performance
- D. To speed up repeated value queries on an attribute, a client can construct and retain a UsdAttributeQuery or UsdResolveInfo
- E. Composition (the prim index) is cached for fast access; the USD core does not pre-compute or cache per-composed-property values

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 55

Domain: Composition

A USD file contains two top-level prims, Car and Building, but its defaultPrim layer metadata is not set. Another layer references this file without specifying an explicit target prim path. What is the result?

- A. Both top-level prims are grafted into the referencing namespace simultaneously.
- B. USD automatically selects the first top-level prim in declaration order as the implicit default.
- C. The reference resolves as an empty layer and a warning is output to the log.
- D. USD raises a fatal error and refuses to open the referencing stage.

Source: Learn OpenUSD: Default Prim — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/default-prim.html>

Question 56

Domain: Customizing USD (Select two options.)

Which two statements correctly describe how IsA (typed) schemas and API schemas differ in how they are assigned to and identified on a USD prim? (Select two.)

- A. Both IsA and API schemas are applied by modifying the prim's typeName, and the distinction is purely organizational
- B. An IsA schema is assigned by setting the prim's typeName metadata; an API schema is list-edited into the apiSchemas metadata
- C. A prim can hold only one IsA schema at a time, whereas multiple API schemas may be applied to the same prim
- D. An IsA schema is queryable via HasAPI; an API schema is queryable via IsA on the prim
- E. API schemas assign a typeName to a prim and are therefore concrete; IsA schemas are always abstract

Source: Learn OpenUSD: Schemas — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/schemas.html>

Question 57

Domain: Visualization

A scene references an asset and binds the asset's mesh to a material whose prim sits OUTSIDE the asset's defaultPrim hierarchy. What problem is most likely when the asset is referenced into another stage?

- A. The binding strength is demoted to weakerThanDescendants, because out-of-hierarchy targets lose precedence
- B. The binding relationship is discarded during composition, because relationship targets do not span reference arcs
- C. The binding does not resolve, because the material prim is outside the encapsulated hierarchy that gets referenced
- D. The material is copied under the defaultPrim, because references pull out-of-hierarchy prims into the target stage

Source: Learn OpenUSD: What Is Encapsulation? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/what-is-encapsulation.html>

Question 58

Domain: Customizing USD (Select three options.)

A team is choosing among the native OpenUSD file formats. Which statements are correct?

- A. The four native formats are the complete set of file formats that USD stages can read; other 3D formats require conversion to USD first
- B. USDA is the recommended production format for large data-heavy geometry layers because text opens faster than binary
- C. USDC (Crate) is a compressed binary format optimized for fast loading and heavy numeric data, while USDA is human-readable ASCII
- D. USDZ is an atomic, uncompressed zipped archive intended as read-only packaging/delivery, ideal for XR
- E. A .usd file may be either ASCII or binary, so the underlying encoding can change without breaking references

Source: Learn OpenUSD: OpenUSD File Formats — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/usd-file-formats.html>

Question 59

Domain: Composition

A variant set is designed for real-time toggling. Which type of variant operation is considered lightweight because it leaves the composition index unchanged?

- A. Introducing new sublayer arcs that add additional layers to the layer stack.
- B. Swapping which referenced layer file is pulled into the scene.
- C. Changing activation opinions that determine which prims are composed onto the stage.
- D. Changing an attribute value, such as switching a color from one value to another.

Source: Learn OpenUSD: Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/variant-sets/what-are-variant-sets.html>

Question 60

Domain: Customizing USD

Under the USD per-schema versioning proposal, a second major version of the SphereLight schema is created. How is that version encoded in the schema identifier, and what is the identifier for version 0?

- A. Version 2 is encoded as 'SphereLight_2'; version 0 is encoded as 'SphereLight_0'
- B. Version 2 is encoded as 'SphereLight@2'; version 0 is encoded as 'SphereLight' with no suffix
- C. Version 2 is encoded as 'SphereLight_2'; version 0 is encoded as 'SphereLight' with no suffix
- D. Version 2 is encoded as 'SphereLight.v2'; version 0 is encoded as 'SphereLight.v0'

Source: OpenUSD: Schema Versioning — https://openusd.org/release/wp_schema_versioning.html

Answer Key & Explanations

1. **Answer: B, C, E** [Composition ▪ obj variant capabilities — what a variant can author]

The source explicitly lists three things a variant can do: 'Variants can sparsely override properties on the prim or any descendant prim,' 'Variants can define new descendant prims,' and 'Variants can be used to author new composition arcs that would only exist within that variant.' Permanently removing external composition arcs is not a variant capability — variants are scoped to their own hierarchy. Renaming a prim path is not a variant capability; that requires the relocates arc.

Source: Learn OpenUSD: Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/variant-sets/what-are-variant-sets.html>

2. **Answer: A** [Data Exchange ▪ obj File format selection for asset delivery]

The corpus states that USDZ 'is an atomic, uncompressed, zipped archive so that we can deliver all of the necessary assets together,' making it the correct format for complete, portable delivery. USDA is optimal for small, human-readable interface layers — not bundled delivery. USDC prioritizes loading performance for data-heavy layers but does not bundle textures. The generic USD extension offers flexibility between ASCII and binary but provides no bundling capability.

Source: Learn OpenUSD: OpenUSD File Formats — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/usd-file-formats.html>

3. **Answer: A** [Composition ▪ obj payloads — deferred loading and strength relative to references]

The source states: 'Payloads are like references, but with the added ability of being able to load and unload payloads on demand' and 'Payloads are weaker than references in the composition strength order (LIVERPS).' Choices with eager loading invert the defining characteristic of payloads. The choice with deferred loading but stronger strength inverts the LIVERPS ordering.

Source: Learn OpenUSD: Payloads — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/what-are-payloads.html>

4. **Answer: C, D, E** [Pipeline Development ▪ obj workstreams — layer stack design and avoiding procedural growth]

The guidance states that each workstream should only author its unique contributions sparsely, rather than needlessly overwriting work done by another workstream. It also recommends keeping layer stacks simple and manageable because there is a cost associated with resolving and opening each layer. Splitting geometry and shading into separate sublayered workstream layers lets one content creator work on geometry while another iterates on materials independently. The option endorsing procedurally growing layer stacks contradicts the explicit warning that layer stacks are not a replacement for asset versioning systems. The option declaring computational workstream layers must be static invents a restriction not present in the guidance.

Source: Learn OpenUSD: Modeling Workstreams With Layer Stacks — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/workstreams/modeling-workstreams.html>

5. **Answer: C** [Composition ▪ obj inherit arcs — broadcast behavior across layer stacks]

The source states: 'The broadcast only applies to the context in which it is authored. The opinion authored in Scenario_A doesn't affect Scenario_B.' Overrides on an unencapsulated class in a downstream stage are scoped to that stage. The global-propagation distractor conflates overriding the class in the asset file itself with overriding it in a downstream layer. The ignore-stronger distractor reverses how LIVERPS works. The convert-to-def distractor invents a specifier mutation that does not occur.

Source: Learn OpenUSD: Inherits — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/inherits-specializes/what-is-inherits.html>

6. **Answer: D** [Content Aggregation ▪ obj scenegraph instancing — hierarchical primvar refinement without new prototypes]

Hierarchical primvar refinement works by authoring primvar opinions on the instanceable prim itself. The subgraphs among the instances remain identical, so no new prototypes are created. The single prototype handles all the shading variability because primvars inherit down the prim hierarchy into the prototype's shader. The option citing one prototype

per distinct value confuses primvar refinement with variant-set refinement. The option describing silent merging invents an internal mechanism. The option about a shared prototype for the modified group applies a deinstancing-style logic that does not match primvar behavior.

Source: Learn OpenUSD: Hierarchical Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-hierarchical-refinement.html>

7. **Answer: B** [Pipeline Development ■ obj organizing prim hierarchy — Geometry and Looks scope conventions]

The exercise page states the goal directly: organizing mesh prims under a Geometry scope and material prims under a Looks scope 'will provide organization and define jurisdictions for products contributed by modelers versus those contributed by surfacers.' This separation makes it clear to downstream users how prims relate and who is responsible for each area. The option about kind metadata boundaries misapplies the model-hierarchy traversal concept to organizational scopes. The option about satisfying encapsulation conflates scope organization with the encapsulation rule about keeping bindings within the defaultPrim. The option about marking scopes as components inverts the role of Scope prims, which carry no kind by default.

Source: Learn OpenUSD: Exercise: Organizing Prim Hierarchy — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-structure-principles/organizing-prim-hierarchy.html>

8. **Answer: A** [Debugging and Troubleshooting ■ obj customData dictionary resolution — per-key merging]

The source states: 'Dictionaries combine element by element, so if one layer has customData[keyOne] and another has customData[keyTwo], the final result will have both keys.' For shared keys the strongest opinion wins. Therefore 'source' resolves from the stronger layer, 'status' (unique to the stronger layer) is retained, and 'owner' (unique to the weaker layer) is also retained. The winner-takes-all distractor misapplies the strongest-wins rule to the whole dictionary. The weakest-accumulates distractor inverts opinion strength. The error distractor invents a restriction.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

9. **Answer: D** [Customizing USD ■ obj schema versioning — criteria for when to version]

The corpus lists 'Changing the fallback value for an attribute that would have an impact' under 'Do Version a Schema When...'. Adding an attribute with a no-effect fallback, adding a new allowed token, and changing displayName or doc are all listed under 'Do not Version a Schema When...' because they do not cause behavioral changes to existing assets.

Source: OpenUSD: Schema Versioning — https://openusd.org/release/wp_schema_versioning.html

10. **Answer: A** [Composition ■ obj 1.10]

A prim is encapsulated when it is a descendant of the referenced root prim. When 'asset.usd' is referenced using 'World' as the source prim (or as the defaultPrim), only the subtree rooted at 'World' is grafted into the new scene. The 'Looks' scope at the same level as 'World' is outside that subtree and therefore not brought along. The material binding relationship now points to a path that does not exist in the referencing context, so the binding is silently dropped.

Source: Learn OpenUSD: What Is Encapsulation? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/what-is-encapsulation.html>

11. **Answer: D** [Debugging and Troubleshooting ■ obj composition caching vs value resolution — key distinction]

The source states: 'Composition is cached, value resolution is not. When you open a stage or add new scene data, USD creates an index of the composition logic and result at the prim-level for quick access. However, USD doesn't pre-calculate the final values of properties.' The both-cached distractor conflates the two systems. The neither-cached distractor contradicts the explicit statement that composition is indexed. The value-cached distractor inverts which system is cached.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

12. **Answer: B** [Composition ■ obj class specifier — rendering and traversal behavior]

The source states: 'Hydra does not image (or render) a prim with a class specifier or any of its descendants' and 'Prims that resolve to class specifiers ... won't be visited by default stage traversals, meaning it will be ignored by traversals such as those used for rendering.' The reduced-resolution distractor fabricates a rendering mode. The double-visit distractor describes the PreAndPostVisit traversal mode, which is unrelated to the specifier. The read-only distractor

confuses class with immutability.

Source: Learn OpenUSD: Specifiers — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/specifiers.html>

13. **Answer: D** [Content Aggregation ▪ obj point instancing — invisiblelds vs inactivelds semantics]

The page states the rule directly: if you need to animate an instance on and off over time, use invisiblelds; otherwise use inactivelds for permanent pruning. Additionally, inactivelds supports sparse list-editing so you can add and remove ids without reauthoring the full array. The option swapping the two attributes inverts the stated rule. The option using protoIndices for removal misapplies the prototype-mapping attribute. The option using the visibility attribute applies the prim-level visibility concept rather than the PointInstancer-specific mechanism.

Source: Learn OpenUSD: Refining Point Instances — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-point-instances.html>

14. **Answer: A, D, E** [Data Exchange ▪ obj Benefits of the data extraction phase]

The corpus lists three explicit benefits of data extraction: (1) Output OpenUSD data looks like it did in the source format, making it easier to understand and debug — matching the legibility option; (2) It simplifies round-tripping of data — matching the round-trip option; (3) It provides a common foundation for all data transformation in the transform phase — matching the common-foundation option. The option describing user export options as an extraction-phase activity contradicts the corpus, which explicitly states that export options are data transformation and not part of extraction. The option claiming automatic defaultPrim and upAxis resolution describes transform-phase chasers, not the extraction phase.

Source: Learn OpenUSD: What Is Data Extraction? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/what-is-data-extraction.html>

15. **Answer: B, E** [Data Modeling ▪ obj 5.5]

Usd.TimeCode.EarliestTime() returns the first authored time sample when animation exists, and Get() with no time code is equivalent to Get(Usd.TimeCode.Default()), returning the authored default. The claim that a bare Get() returns the first time sample inverts that behavior; unauthored attributes resolve to the schema fallback, not to values from ancestor prims; and value resolution is not cached the way composition is.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

16. **Answer: D, E** [Pipeline Development ▪ obj 7.6]

The guidance is to keep layer stacks simple and manageable, avoiding workstreams that might grow procedurally over time because each layer carries a resolve-and-open cost, and to have each workstream author only its unique contributions sparsely rather than needlessly overwriting another workstream's work. Layer stacks are explicitly not a replacement for asset versioning systems, so capturing versions as sublayers is wrong. Simple assets can live in a single layer, so pre-splitting them is wrong. And computational workstreams can be dynamic, not necessarily consistent between evaluations.

Source: Learn OpenUSD: Modeling Workstreams With Layer Stacks — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/workstreams/modeling-workstreams.html>

17. **Answer: A** [Composition ▪ obj specializes — fallback value semantics]

The source states: 'In OOP, objects that override their class members will ignore updates made to those class members.' A prim that individually authors its own opinion is stronger than the specializes fallback under LIVERPS. The always-propagates distractor incorrectly describes specializes — only prims without their own opinion pick up the updated fallback. The retain-original distractor invents a snapshot behavior. The conflict-to-schema distractor invents a resolution mode.

Source: Learn OpenUSD: Specializes — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/inherits-specializes/what-is-specializes.html>

18. **Answer: C** [Content Aggregation ▪ obj 2.1]

The lesson states: "You tell OpenUSD the candidates for instancing, it figures out the prototypes implicitly from the authored composition arcs." Setting instanceable = true is the explicit signal; here the shared reference to RobotArm.usd yields one implicit prototype with two instances.

Source: Learn OpenUSD: Introduction to Scenegrph Instancing — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/>

19. **Answer: B** [Composition ▪ obj encapsulation — material binding behavior when source prim is unencapsulated]
The source states: 'Unlike our encapsulated example, the scope called Looks is not encapsulated inside of World. So when we reference unencapsulated.usd, the cube will be brought over but the material binding will no longer work because the red material would not exist in the new context.' Automatic path remapping for sibling scopes does not exist. Promotion to the root layer is not a USD behavior. Corpus sources note a warning is emitted, so the silent-discard distractor is also incorrect.
Source: Learn OpenUSD: What Is Encapsulation? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/what-is-encapsulation.html>
20. **Answer: C** [Pipeline Development ▪ obj model hierarchy — component kind rules and gprim tagging]
The page states explicitly: 'A gprim tagged as a component is a sign that a model hierarchy is deep.' The model hierarchy is designed to prune traversal at the component model boundary, and having that boundary pushed all the way down to a gprim indicates the hierarchy is excessively deep, introducing unnecessary composition costs. The option about component-within-component misapplies the descendant-component rule to a Mesh itself. The option about typeName conflicts invents a constraint not in the source. The option requiring all ancestors to carry component inverts the actual rule, which requires ancestors to carry group or assembly.
Source: Learn OpenUSD: Model Hierarchy Considerations — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/model-hierarchy-considerations.html>
21. **Answer: C** [Data Exchange ▪ obj 4.6]
UsdPreviewSurface is authored through UsdShade and bound to geometry using UsdShadeMaterialBindingAPI. UsdGeom is for geometry, not shaders; UsdSkelBindingAPI is for skeletal binding; and bindings are explicit relationships, not inferred from namespace proximity. Legacy shininess values are converted to roughness rather than authored raw.
Source: Learn OpenUSD: Exercise: Extracting Materials — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-materials.html>
22. **Answer: D** [Pipeline Development ▪ obj 7.5]
UsdStage.Flatten composes the entire stage into a single layer, removing composition arcs and any proprietary file-format dependencies, and is distinct from flattening one layer stack (UsdUtilsFlattenLayerStack), which merges just the sublayers of a layer stack. The option crediting UsdUtilsFlattenLayerStack with whole-stage composition has the two operations swapped. Setting the edit target only chooses where future opinions are authored and bakes nothing. SdfChangeBlock batches change notifications during authoring and never merges arcs or layers.
Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>
23. **Answer: B** [Composition ▪ obj references — grafting vs sublayering for multiple copies]
The source states: 'Sublayering a layer multiple times won't work in this case ... the same sphere would just be redefined three times because there is no way to specify that the copies of the sphere should be defined at different paths.' Sublayers add content without path remapping, so distinct sibling paths cannot be produced. The error-on-duplicate distractor invents a constraint that does not exist. The no-definitions distractor incorrectly restricts sublayer specifiers. The merged-opinion distractor misdescribes sublayer semantics.
Source: Learn OpenUSD: References — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/what-are-references.html>
24. **Answer: A** [Visualization ▪ obj UsdLuxDistantLight default emission axis]
The corpus states: 'A light is emitted from a distance source along the -Z axis.' This is the defined default emission direction for UsdLuxDistantLight before any transforms are applied. The rotation shown in the snippet rotates the light but does not alter which axis is the default emission axis.
Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>
25. **Answer: A, D, E** [Content Aggregation ▪ obj broadcasted refinement — inherit/specialize arc broadcasting and prototype creation]

The source data shows broadcasted refinement raised the prototype count from 3 to 4 — a new prototype was created for the affected boxes because their composed subgraph now differs from unmodified boxes. The instance count remained at 1450, confirming instancing stayed active on all boxes. The prim count rose from 1711 to 1747, confirming that the new arc and its referenced content introduce additional prims. The option denying prototype creation confuses broadcasted refinement with hierarchical primvar refinement, which does not generate new prototypes. The option claiming the total prim count decreases is contradicted by the recorded increase of 36 prims.

Source: Learn OpenUSD: Broadcasted Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-broadcasted-refinement.html>

26. **Answer: C** [Data Exchange ▪ obj Two-phase data exchange approach]

The corpus states: 'The goal is to translate the data to OpenUSD as directly as possible to maintain fidelity to the source format. Try to map concepts from the source format to concepts in OpenUSD as much as possible to preserve the integrity and structure of the source data.' Applying export options and restructuring content are explicitly described as transform-phase activities. Pruning irrelevant data and optimization are also transform-phase responsibilities. Validation against compliance rules is the role of asset validation tools such as usdchecker, not the extract phase.

Source: Learn OpenUSD: What Is Data Exchange? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-exchange/what-is-data-exchange.html>

27. **Answer: C** [Pipeline Development ▪ obj reference/payload pattern — purpose of lofting fields above the payload]

The page states that 'lofting fields can help avoid the need to load a payload in certain contexts, thereby improving overall performance.' By authoring the primvar or variant set above the payload on the interface layer, consumers can discover and interact with these parameters without triggering payload loading. The exercise confirms that with the payload unloaded the primvars:accentColor attribute is still listed in the Properties panel. The option about removing field opinions contradicts the page, which says the lofted elements still advertise the sets and selections. The option about root-layer promotion mischaracterizes what lofting does. The option about global metadata invents a metadata conversion step.

Source: Learn OpenUSD: Exercise: Lofting Primvars — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/reference-payload-pattern/lofting-primvars.html>

28. **Answer: B** [Composition ▪ obj 1.4]

A reference grafts only the targeted source prim's subtree, so targeting Cube brings the geometry but not the material defined under World; the binding's target falls outside the scope of the reference, and USD warns and ignores it. The reference still resolves even though Cube is not the defaultPrim, because an explicit prim path is supplied. Relationship targets are not re-pathed to chase out-of-scope prims, so the material does not follow the Cube across. And grafting never drops geometry in favor of a relationship target.

Source: Learn OpenUSD: Exercise: Experimenting With Encapsulation — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/experimenting-with-encapsulation.html>

29. **Answer: B** [Data Exchange ▪ obj Reusability of transformation versus extraction code]

The corpus states: 'While extraction code from source formats to OpenUSD is largely unique, data transformation for specific clients or workflows can be designed for reuse across multiple exporters.' This is because transformation operates on OpenUSD data — 'Export options, optimizations and asset structure code can be shared across applications since they all operate on OpenUSD data.' The option claiming both phases are unique ignores the stated reusability of transformation code. Swapping the direction — claiming extraction is reusable — contradicts the corpus directly. The option claiming both are reusable because both operate on OpenUSD data misattributes where OpenUSD data enters the pipeline; only transformation operates on already-extracted OpenUSD data.

Source: Learn OpenUSD: What Is Data Transformation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/what-is-data-transformation.html>

30. **Answer: D** [Debugging and Troubleshooting ▪ obj active/inactive prims — effect on traversal and descendants]

The source states: 'The prim itself will be excluded from default stage traversals' and 'All scene description for the deactivated prim's descendants will not be composed onto the stage.' Both /World/Box and all of its descendants are excluded. The descendants-still-visited distractor incorrectly suggests children are independently traversable after a

parent is deactivated. The empty-values distractor misdescribes the effect as attribute-only rather than a composition exclusion. The exception distractor confuses deactivation with deletion.

Source: Learn OpenUSD: Active and Inactive Prims — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/active-inactive-prim.html>

31. **Answer: A** [Data Modeling ▪ obj primvar interpolation — constant mode]

The 'constant' interpolation mode supplies exactly one value that applies to the whole prim. 'uniform' requires one value per face (2 for this mesh), 'vertex' requires one per unique point (6), and 'faceVarying' requires one per face-vertex entry (8).

Source: Learn OpenUSD: Primvars — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/primvars.html>

32. **Answer: B** [Debugging and Troubleshooting ▪ obj attribute value resolution — Get() default vs time-sampled values]

The source states: 'Get() is the same as Get(Usd.TimeCode.Default())' which returns the authored default value (1.5, 1.5, 1.5). Querying before the first time sample is clamped to the first sample: 'Scale - AnimCube at t=1 (before first sample, clamped): (2.5, 2.5, 2.5)'. Therefore Get() returns the default and Get(Usd.TimeCode(1)) returns the clamped first sample. The swapped distractor reverses which call returns which value. The both-time-sampled distractor ignores the default value. The both-default distractor ignores the clamping behavior for pre-sample queries.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

33. **Answer: C** [Data Modeling ▪ obj metersPerUnit fallback value]

The corpus states: 'If metersPerUnit is not explicitly authored in a USD file, the fallback value is 0.01 (centimeters)'. The other values represent meters, millimeters, and feet — each a real unit but not the USD default.

Source: Learn OpenUSD: Units in OpenUSD — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/units.html>

34. **Answer: D** [Data Exchange ▪ obj 4.4]

Extraction code from a source format to OpenUSD is largely unique, while export options, optimizations, and asset-structure transformation code can be shared across exporters because it all operates on OpenUSD data. The reversed claim swaps the two, and the all-unique and all-reusable variants each contradict half of the documented split.

Source: Learn OpenUSD: What Is Data Transformation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/what-is-data-transformation.html>

35. **Answer: C** [Visualization ▪ obj UsdLux API schema adding light behavior to mesh prims]

The corpus states: 'UsdLux has API schemas that allow you to add light behavior to prims in your scene, so you can also add light properties to meshes and volumes.' An API schema such as MeshLightAPI is applied to the existing mesh prim. Defining a child light prim, setting purpose, or wiring a shader connection are different USD mechanisms that do not achieve mesh-light emission on the same prim.

Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>

36. **Answer: B** [Content Aggregation ▪ obj scenegraph instancing — instanceable metadata and composition arcs]

The page explains this as a deliberate 'trick': setting instanceable = true is an explicit signal, but OpenUSD derives prototypes from composition arcs. Without any composition arcs, there is nothing for the engine to derive prototypes from, so no prototypes are created and memory usage is unchanged. The option stating one prototype is generated from the local layer stack misapplies local-opinion logic; the option claiming three prototypes are always generated ignores the composition-arc requirement; the option restricting instanceable to assembly prims invents a kind constraint that does not exist.

Source: Learn OpenUSD: Introduction to Scenegraph Instancing — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/authoring-scenegraph-instancing/scenegraph-instancing-intro.html>

37. **Answer: D** [Composition ▪ obj sublayer opinion strength ordering]

The source states 'sublayerB comes before sublayerA, meaning the authored opinions in sublayerB will outweigh sublayerA's opinions.' The first-listed sublayer is the stronger one. The last-listed distractor inverts this ordering. The root layer option is only correct when the root layer itself authors an opinion. The blending distractor describes a behavior

that does not exist in USD composition.

Source: Learn OpenUSD: Sublayers — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/sublayers/what-are-sublayers.html>

38. **Answer: B** [Content Aggregation ▪ obj scenegraph instancing — prototype creation rules with variant selections]

Variant sets are part of composition, and variant selections are evaluated when OpenUSD determines what implicit prototypes to create. Because the second prim has a different variant selection, its composed subgraph differs from the first prim's, so OpenUSD generates a separate prototype for it. The option claiming one prototype ignores that variant selections alter the composition arcs used for prototype derivation. The option claiming each instanceable prim always gets its own prototype contradicts how shared-reference instancing works. The option stating variant selections prevent prototype sharing is an invented constraint.

Source: Learn OpenUSD: Refinement Using Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-variant-set-refinement.html>

39. **Answer: B** [Data Exchange ▪ obj Unit metadata reconciliation during composition]

The corpus states: 'A 6-meter tall tree (metersPerUnit = 1.0) referenced into a centimeter-based stage (metersPerUnit = 0.01) will compose as 6 centimeters tall by default—100 times too small.' USD does not perform automatic unit conversion for metersPerUnit — geometric values are 'copied literally without unit conversion.' The distractor describing automatic scaling contradicts the stated behavior. Refusing to compose is not a documented behavior. The 100-times-too-large outcome is a direction flip; the correct direction is too small because a meter-scale number is interpreted in centimeter units.

Source: Learn OpenUSD: Units in OpenUSD — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/units.html>

40. **Answer: C** [Data Exchange ▪ obj Asset validation tooling — usdchecker and Omniverse Asset Validator]

The corpus states that Omniverse Asset Validator 'is based on usdchecker and provides a GUI for asset validation' and lists its specific capabilities: 'User interface, Command line interface, Python API, Automatic fixes for failed validations, Ability to add new rules.' usdchecker is the underlying command-line tool that validates stages and USDZ packages but does not offer a Python API for custom rules or automatic fixes. usdcat is a conversion and inspection utility, not a validation framework. UsdGeomBBBoxCache is a USD API for computing geometry bounding boxes, not an asset validation tool.

Source: Learn OpenUSD: What Is Asset Validation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/asset-validation/what-is-asset-validation.html>

41. **Answer: D** [Pipeline Development ▪ obj asset encapsulation — material bindings outside the defaultPrim hierarchy]

The exercise shows the exact USD warning message: the relationship target 'refers to a path outside the scope of the reference ... Ignoring.' Only prims defined beneath the defaultPrim hierarchy are brought along when the asset is referenced, so a material at a sibling path to World is left behind. The option about silent remapping invents a resolver behavior. The option claiming references pull the entire layer contradicts the encapsulation scoping rule. The option making payload a factor introduces a condition not in the source.

Source: Learn OpenUSD: Exercise: Encapsulating Your Asset — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-structure-principles/encapsulating-your-asset.html>

42. **Answer: C** [Debugging and Troubleshooting ▪ obj traversal predicates — correct bitwise combination syntax]

The source includes an explicit caution: 'Boolean operators, like and/or/not, will NOT combine predicates as intended. Always combine traversal predicates with bitwise operators (&, |, ~).' The correct form is 'predicate = Usd.PrimIsActive & Usd.PrimIsLoaded'. The union distractor incorrectly frames the failure as an intersection-vs-union issue. The __bool__ distractor describes a plausible Python mechanism but is not the precise documented reason. The short-circuit distractor describes standard Python and short-circuiting but is not the documented cause.

Source: Learn OpenUSD: Stage Traversal — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/stage-traversal.html>

43. **Answer: A** [Composition ▪ obj LIVERPS strength order — inherits vs references vs specializes]

The LIVERPS page lists the strength order from strongest to weakest: local, inherits, variant sets, references, payloads, specializes. Inherits wins over references and specializes when no local opinion is present. The specializes distractor

correctly identifies specializes as a fallback but mistakes its position. The 'most common arc' distractor misapplies a frequency claim to strength. The averaging distractor describes a behavior USD does not implement.

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

44. **Answer: D** [Data Modeling ▪ obj timeCodesPerSecond real-world time mapping]

The corpus states: 'if the root layer has timeCodesPerSecond=24, a time code value of 48.0 would correspond to 2 seconds (48/24) of real time after time code 0.' The other options apply incorrect ratios or invert the formula.

Source: Learn OpenUSD: Time Codes and Time Samples — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/timcodes-timesamples.html>

45. **Answer: B** [Data Modeling ▪ obj custom=True flag semantics]

The corpus states explicitly: 'Setting custom=True has no impact on the way that OpenUSD treats the attribute, but it is a useful piece of metadata to inform end-users and developers that a particular attribute is not part of an existing schema.' The other options describe behaviors the flag does not provide.

Source: Learn OpenUSD: Custom Properties — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/custom-properties.html>

46. **Answer: C** [Composition ▪ obj 1.7]

LIVERPS orders opinions Local, Inherits, Variants, rLocates, References, Payloads, Specializes. With no local opinion, the inherited opinion is the strongest remaining and wins over both the variant and the reference. References resolve after inherits, not before; variant selections sit below inherits in strength rather than overriding them; and strength is decided by arc class first and then order within a class, never by which sublayer is strongest irrespective of arc type.

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

47. **Answer: A** [Visualization ▪ obj HdStorm — out-of-the-box real-time render delegate]

The corpus states: 'OpenUSD ships with HdStorm, the real-time OpenGL/Metal/Vulkan render delegate leveraged by usdview and many other tools. It also ships with HdTiny and HdEmbree, which can be used as examples of how to implement render delegates.' HdPrman is a third-party plugin, not shipped in core OpenUSD.

Source: Learn OpenUSD: Hydra — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/hydra.html>

48. **Answer: D** [Visualization ▪ obj Hydra architecture — three main parts]

The corpus states: 'A scene delegate, which provides the scene information; The render index, which keeps track of changes and manages the scene; The render delegate, which uses the render index and scene delegate to visualize scene information to create the final image.' The other options swap roles, rename parts, or use non-USD terminology.

Source: Learn OpenUSD: Hydra — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/hydra.html>

49. **Answer: D** [Data Exchange ▪ obj File format plugins as USD integration mechanism]

The corpus states that file format plugins allow 'An OpenUSD stage can include a source file format directly as a reference, payload or sublayer. The source file format is translated on the fly, while it is read as a USD document,' and crucially that 'The source file format can remain the source of truth.' Standalone converters produce a separate output file and require a pre-export step. DCC importers translate from OpenUSD into the application's runtime format — the opposite direction. DCC exporters translate from the application's runtime format to OpenUSD, also requiring a pre-export step and not retaining the original format as the source of truth.

Source: Learn OpenUSD: What Is Data Exchange? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-exchange/what-is-data-exchange.html>

50. **Answer: D, E** [Data Modeling ▪ obj automatic vs manual unit reconciliation]

The corpus states timeCodesPerSecond is automatically reconciled during composition: 'USD automatically scales time samples so animations play back correctly relative to the root layer's timeCodesPerSecond.' The same automatic scaling applies through layer offsets. metersPerUnit, upAxis, and kilogramsPerUnit are listed as 'Manually Handled' — the assembler is responsible for corrective transforms.

Source: Learn OpenUSD: Units in OpenUSD — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/units.html>

51. **Answer: A** [Data Modeling ■ obj custom attribute namespace convention for grouped data]

The corpus states: 'Since USD doesn't have a native struct type, namespace-prefixed attributes serve as the standard convention for representing this kind of grouped data.' The example demonstrates `acme:sensor:temperature / humidity / pressure`. `customData` is a composable dictionary described as more costly for composition; a `Scope` child adds unnecessary hierarchy; a string attribute loses type safety.

Source: Learn OpenUSD: Custom Properties — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/custom-properties.html>

52. **Answer: D** [Pipeline Development ■ obj model hierarchy — assembly and component kind relationships]

The page states that 'component models cannot contain other component models as descendants.' The component kind denotes a leaf asset or pruning point in the model hierarchy. Descendants can use subcomponent for important prims that are outside the model hierarchy, but another component kind as a descendant is forbidden. The option requiring all descendants to be subcomponent overstates the rule. The option permitting group descendants below a component inverts the hierarchy direction — groups and assemblies are ancestors, not descendants. The option permitting assembly descendants is also contradicted by the leaf-boundary rule.

Source: Learn OpenUSD: What Are Model Kinds? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/what-are-model-kinds.html>

53. **Answer: B** [Pipeline Development ■ obj asset parameterization — variant sets vs primvars and scenegraph instancing]

The page states directly: 'Variations in variant selections will generate new prototypes for downstream contexts, while primvars will not.' This makes parameterization through primvars the lighter choice for single property parameters, offering upfront memory savings at the cost of additional lookups in materials. The option inverting the two behaviors misattributes prototype generation to primvars. The option claiming both generate prototypes per value is contradicted for primvars. The option claiming both share a single prototype is contradicted for variant selections.

Source: Learn OpenUSD: What Is Asset Parameterization? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-parameterization/what-is-asset-parameterization.html>

54. **Answer: D, E** [Debugging and Troubleshooting ■ obj 6.1]

Composition is cached as a prim-level index for fast access, while the USD core does not pre-compute or cache per-composed-property values, and clients can construct and retain a `UsdAttributeQuery` or `UsdResolveInfo` to speed repeated queries on an attribute. `GetPropertyStack` is documented for debugging only, not as a resolution path. How data combines during value resolution is specific to each metadatum and varies by data type — dictionaries, for example, merge element by element — so it is not identical across metadata. And it is composition that is internally multi-threaded; value resolution is designed to be efficiently multi-threaded by clients, not scaled by opening many stages in parallel.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

55. **Answer: C** [Composition ■ obj defaultPrim — behavior when unset during referencing]

The source states: 'If no `defaultPrim` is set then when the above `.usda` is brought in as a payload or reference it will resolve as an empty layer and output a warning message in the log.' The fatal-error distractor overstates severity. The both-prims distractor misapplies multi-prim grafting, which requires explicit authoring. The first-prim distractor invents an implicit selection rule that does not exist.

Source: Learn OpenUSD: Default Prim — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/default-prim.html>

56. **Answer: B, C** [Customizing USD ■ obj IsA vs API schema — assignment and identification]

The corpus states: 'We use the `typeName` metadata to assign an IsA schema to a prim' and 'each prim can only subscribe to one IsA schema at a time.' API schemas 'do not assign a `typeName` to a prim. Instead, are list-edited in the `apiSchemas` metadata.' The second option swaps the query methods; the fourth inverts the concreteness rule; the fifth is incorrect because `typeName` exclusively identifies IsA schemas.

Source: Learn OpenUSD: Schemas — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/schemas.html>

57. **Answer: C** [Visualization ■ obj 8.3]

A reference composes the defaultPrim's subtree, so a material prim outside that encapsulated hierarchy does not exist in the new context and the binding cannot resolve. The relationship itself is still composed rather than discarded, references do not copy out-of-hierarchy prims into the target stage, and binding strength is authored metadata that composition never demotes.

Source: Learn OpenUSD: What Is Encapsulation? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/what-is-encapsulation.html>

58. **Answer: C, D, E** [Customizing USD ■ obj 3.5]

USDC is the compressed binary Crate format suited to fast loading and numerically heavy data while USDA is human-readable ASCII; USDZ is an atomic, uncompressed zipped archive meant as read-only packaging and delivery, ideal for XR; and a .usd file can be either encoding, so it can change without breaking references. The completeness claim is wrong because any other 3D format can be loaded through file format plugins without prior conversion. The USDA-for-heavy-geometry claim is wrong because binary crate, not ASCII text, is preferred for large geometry layers.

Source: Learn OpenUSD: OpenUSD File Formats — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/usd-file-formats.html>

59. **Answer: D** [Composition ■ obj variant operations — lightweight vs heavyweight switches]

The source states that lightweight variant operations include 'Changing attribute values - For example, switching a color from red to blue' because 'USD doesn't need to rebuild the composed scene structure. The stage's composition index remains the same.' Swapping referenced layers is explicitly listed as heavyweight because it changes the composition structure. Changing activation opinions is also listed as heavyweight. Introducing new sublayer arcs would change composition structure.

Source: Learn OpenUSD: Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/variant-sets/what-are-variant-sets.html>

60. **Answer: C** [Customizing USD ■ obj schema versioning — version identifier encoding]

The corpus states: 'The schema version will be embedded in the schema identifier itself as a suffix, e.g. "SphereLight_2" is version 2 of the SphereLight schema.' It further states: 'Version 0 of a schema will be represented without a version suffix. For example "SphereLight" is version 0 of SphereLight; "SphereLight_0" will not be allowed as a schema identifier.' The underscore-zero form and other separator styles are therefore incorrect.

Source: OpenUSD: Schema Versioning — https://openusd.org/release/wp_schema_versioning.html